

Ego4D-LiteSTA Configuration System

This document describes the new YAML-based configuration system that replaces in-code toggles with external, version-controlled configuration files.

Table of Contents

- 1. [Overview](#)
- 2. [Directory Structure](#)
- 3. [Configuration Files](#)
- 4. [Core Modules](#)
- 5. [Run Logging](#)
- 6. [Smoke Tests](#)
- 7. [Usage Guide](#)
 - [Local Usage](#)
 - [Google Colab Usage](#)
- 8. [Configuration Reference](#)
- 9. [Migration Guide](#)

Overview

The new configuration system provides:

- **Externalized settings:** All toggles moved from Python code to YAML files
- **Config inheritance:** Base configs shared across tracks via `_base_` key
- **Variable interpolation:** Use `${paths.x}` syntax to reference other config values
- **Environment auto-detection:** Automatically detects Local vs Colab environments
- **CLI/env overrides:** Override any setting via command line or environment variables
- **Smoke tests:** Quick validation with pycache checking for each track

Benefits

Before (In-Code Toggles)	After (YAML Configs)
Edit Python files to change settings	Edit YAML files (no code changes)
Settings scattered across files	Centralized in <code>configs/</code> directory
Hard to version control settings	YAML files are git-friendly
No validation before running	Smoke tests catch issues early
Manual path switching for Colab	Automatic environment detection

Directory Structure

```

local_extraction/
├── configs/                                # YAML configuration files
│   ├── base.yaml                          # Shared settings (paths, runtime, logging)
│   ├── trackA.yaml                       # Track A: Detector configs
│   ├── trackB.yaml                       # Track B: Fusion/Head configs
│   ├── trackC.yaml                       # Track C: RGTP pruning configs
│   └── extraction.yaml                   # Frame/label extraction configs
├── core/                                  # Core modules
│   ├── __init__.py                       # Package exports
│   ├── config_loader.py                  # YAML loading, interpolation, overrides
│   ├── config_adapter.py                 # Bridge to existing toggle-based scripts
│   ├── paths.py                          # Path resolution, Colab detection
│   └── run_logger.py                     # Run logging, metrics tracking
├── runs/                                  # Run logs and outputs
│   ├── Track_A/                          # Track A runs
│   ├── Track_B/                          # Track B runs
│   └── Track_C/                          # Track C runs
├── tests/                                 # Smoke tests
│   ├── __init__.py
│   ├── smoke_test_trackA.py             # Track A validation
│   ├── smoke_test_trackB.py             # Track B validation
│   └── smoke_test_trackC.py             # Track C validation
└── notebooks/
    └── orchestration.ipynb              # Colab orchestration notebook

```

Configuration Files

base.yaml - Shared Settings

```

version: "v2"

paths:
  local_extraction: "local_extraction"
  extracted_frames: "${paths.local_extraction}/${version}/extracted_frames"
  yolo_labels: "${paths.local_extraction}/${version}/yolo_labels_540"
  manifests: "${paths.local_extraction}/${version}/manifests"
  org_annotations: "${paths.local_extraction}/${version}/org_annotations"
  runs: "${paths.local_extraction}/runs"

runtime:
  device: "auto"          # auto, cuda, cpu
  num_workers: 4
  pin_memory: true

```

```
print_progress: true

demo:
  enabled: false          # Quick test mode
  max_samples: 50
  epochs: 2

logging:
  verbose: true
  save_checkpoints: true
  checkpoint_freq: 5

seed: 42
```

trackA.yaml - Detector Configuration (Two Stages)

```
_base_: base.yaml          # Inherit from base

# Stage A: YOLO detector for candidate generation
stage_a:
  mode: "yolo"              # yolo or oracle
  k: 6                      # top-K candidates
  last_frame_only: true
  max_images: null

  yolo:                     # YOLO-specific settings
    weights: "local_extraction/toolkit_yolo/runs/.../best.pt"
    imgsz: 960
    conf_thresh: 0.05
    nms_iou: 0.45

  oracle:                   # Oracle mode settings
    label_space: "clips"

  output:
    save_per_image_csv: false
    run_prefix: "trackA_stageA"

# Stage B: Head/reasoner for semantic prediction
stage_b:
  candidates_source: null   # null = auto-detect latest Stage A run
  crop_size: [256, 256]
  keep_top_n: null
  eval_with_labels: true
  iou_thresh: 0.5

manifests:
  use_clip_manifest: true
  train: "${paths.manifests_root}/head_train_clip.json"
  val: "${paths.manifests_root}/head_val_clip.json"
```

```
output:
  write_head_train_val: true
  run_prefix: "trackA_stageB"

# K-sweep for recall optimization
k_sweep:
  enabled: false
  k_values: [4, 6, 8, 10, 12, 15]
```

trackB.yaml - Fusion/Head Configuration (Complete)

Track B has multiple config classes that are all covered:

Module	Config Class	YAML Section
trackB_tokenizer.py	TokenizerConfig	model.tokenizer.*
trackB_fusion.py	FusionConfig	model.fusion.*
trackB_head.py	HeadConfig	model.head.*
trackB_train_loader.py	TrainConfig	training.*, multi_task.*, demo.*
trackB_eval.py	EvalConfig	evaluation.*

```
_base_: base.yaml

model:
  # TokenizerConfig settings
  tokenizer:
    backbone: "resnet18"
    pretrained: true
    img_size: 224
    freeze_backbone: true

  # Projector (Linear layer)
  projector:
    in_dim: 512
    out_dim: 256

  # FusionConfig settings
  fusion:
    dim: 256
    layers: 2
    heads: 8
    dropout: 0.1

  # HeadConfig settings
  head:
    dim: 256
```

```
    hidden: 256
    num_classes: 2
    dropout: 0.1

# Multi-task learning
multi_task:
    enabled: true
    predict_noun: true
    predict_verb: true
    predict_ttc: true
    ttc_mode: "reg"          # 'reg' or 'bin'
    loss_weights:
        next_active: 1.5
        noun: 0.25
        verb: 0.25
        ttc: 1.0

# TrainConfig settings
training:
    mode: "main"            # 'main' or 'demo'
    epochs: 20
    batch_size: 8
    lr: 1.0e-3
    min_lr: 1.0e-5
    warmup_epochs: 1.0
    label_smoothing: 0.05
    candidate_limit: 16
    normalize_ttc: true
    save_epoch_checkpoints: true
    save_best_checkpoint: true
    early_stopping:
        enabled: true
        patience: 5
        monitor: "mAP"
    eval_every: 1
    amp: false

# EvalConfig settings
evaluation:
    batch_size: 8
    compute_map: true
    save_overlays: true
    hotspot_priors:
        enabled: true
        alpha: 0.3
    clip_rerank:
        enabled: true
        model: "ViT-B/32"
        weight: 0.3

# Demo mode
demo:
    enabled: false
    steps: 10
```

```

    variant: "loader"

# Data sources
data:
    stageB_run: null
    train_manifest: null
    val_manifest: null

```

Creating Track B Config Classes

```

from core import (
    create_train_config,
    create_fusion_config,
    create_head_config,
    create_tokenizer_config,
    create_trackb_eval_config,
)

# Each returns the appropriate config class with YAML values applied
train_cfg = create_train_config()      # TrainConfig
fusion_cfg = create_fusion_config()    # FusionConfig
head_cfg = create_head_config()        # HeadConfig
tokenizer_cfg = create_tokenizer_config() # TokenizerConfig
eval_cfg = create_trackb_eval_config()  # EvalConfig dict

# With overrides
fusion_cfg = create_fusion_config(overrides={
    'model': {'fusion': {'layers': 4, 'heads': 16}}
})

```

lr: 0.001 min_lr: 0.00001 warmup_epochs: 1.0 label_smoothing: 0.05

multi_task: enabled: true loss_w_next: 1.5 loss_w_noun: 0.25 loss_w_verb: 0.25 loss_w_ttc: 1.0

```

### `trackC.yaml` - RGTP Pruning Configuration

```yaml
base: base.yaml

rgtp:
 enabled: true
 rate: 0.1 # Fraction to prune (0.1 = 10%)
 min_keep: 2 # Never prune below this count
 temporal_decay: 0.6
 importance_method: "attention_rollout"

```

```
logit_fill: -12.0 # Logit for pruned tokens

rate_sweep:
 enabled: false
 rates: [0.0, 0.1, 0.2, 0.3, 0.4, 0.5]

instrumentation:
 measure_latency: true
 measure_vram: true
 measure_flops: true
```

---

## Core Modules

### config\_loader.py

Main configuration loading with these features:

```
from core import load_config

Basic loading
cfg = load_config('trackB')
print(cfg.get('training.epochs')) # 50

With overrides (dict)
cfg = load_config('trackB', overrides={
 'demo': {'enabled': True},
 'training': {'epochs': 5}
})

With CLI overrides (parsed from sys.argv)
python script.py --training.lr=0.0001 --demo.enabled=true
cfg = load_config('trackB', parse_cli=True)

Check environment
from core import is_colab, get_device
print(f"Colab: {is_colab()}, Device: {get_device()}")
```

### paths.py

Centralized path management with property aliases for notebook compatibility:

```
from core import Paths, get_paths

Auto-detect environment
paths = get_paths()
```

```
print(paths.frames_root) # Primary field name
print(paths.extracted_frames) # Alias (for notebook compatibility)
print(paths.manifests) # Alias for manifests_root

Create run directory
run_dir = paths.run_dir('Track_A', 'stageA')
-> local_extraction/runs/Track_A/stageA_20251202_143052

Find latest run (handles multiple formats)
latest = paths.latest_run('A') # Works
latest = paths.latest_run('trackA') # Works
latest = paths.latest_run('Track_A') # Works

Find checkpoint
ckpt = paths.find_checkpoint('trackB_best_*.pt')
```

**Property Aliases** (for notebook compatibility):

Field Name	Alias Property
frames_root	extracted_frames
labels_root	yolo_labels
manifests_root	manifests

config\_adapter.py

Bridge between YAML configs and existing scripts:

```
from core import apply_config_to_module, create_train_config

Apply YAML config to module-level toggles
import trackA_stageA
apply_config_to_module(trackA_stageA, 'trackA', stage='stageA')
Now trackA_stageA.DEMO_MODE, trackA_stageA.K, etc. are set from YAML

Create TrainConfig from YAML
train_cfg = create_train_config()
Returns TrainConfig instance with YAML values applied
```

## Run Logging

The `RunLogger` provides unified logging for all track runs, capturing configuration, metrics, system info, and artifacts.

### Features

- **Unique run IDs:** Timestamp-based IDs for each run
- **Config snapshots:** Full configuration captured at start
- **Git info:** Commit, branch, dirty status (skipped in Colab)
- **System info:** Python version, CUDA, GPU details, Colab detection
- **Metrics tracking:** Log any metrics during/after run
- **Artifact tracking:** Log output files (checkpoints, predictions)
- **Status management:** Track success/failure with error messages

## Basic Usage

```
from core import RunLogger, create_run_logger

Option 1: Direct instantiation
logger = RunLogger(track='trackB', stage='train')

Option 2: Factory function (recommended)
logger = create_run_logger(
 track='trackB',
 stage='train',
 config={'epochs': 20, 'lr': 1e-3}, # Optional initial config
 run_dir=Path('custom/run/dir'), # Optional custom directory
)

Log workflow
logger.log_config({'training': {'epochs': 20}})
logger.log_start()

... do training ...

logger.log_metrics({'mAP': 0.45, 'loss': 0.123})
logger.log_artifacts(['checkpoints/best.pt'])
logger.log_end(success=True)

Save and print summary
logger.save()
logger.print_summary()
```

## Run Log Structure

Each run creates a directory under `runs/Track_X/` :

```
runs/
├── Track_B/
│ ├── trackB_train_20251202_143052/
│ │ ├── run_log.json # Full run log
│ │ └── config_snapshot.yaml # Config at run start
```

**run\_log.json contents:**

```
{
 "run_id": "trackB_train_20251202_143052",
 "track": "trackB",
 "stage": "train",
 "timestamp": "2025-12-02T14:30:52",
 "status": "completed",
 "config": { ... },
 "system": {
 "python_version": "3.11.5",
 "torch_version": "2.1.0",
 "cuda_available": true,
 "gpu_name": "NVIDIA RTX 3080",
 "is_colab": false
 },
 "git": {
 "commit": "abc123def456",
 "branch": "main",
 "dirty": false
 },
 "metrics": {
 "mAP": 0.45,
 "loss": 0.123
 },
 "artifacts": ["checkpoints/best.pt"],
 "duration_seconds": 3600
}
```

## Integration in Track Scripts

All track scripts now integrate `RunLogger` :

```
In trackB_train_loader.py
from core import RunLogger, load_config

def main():
 cfg = load_config('trackB')

 # Create logger
 logger = RunLogger(track='trackB', stage='train')
 logger.log_config(cfg.raw)
 logger.log_start()

 try:
 # Training code...
 metrics = train(cfg)
```

```
 logger.log_metrics(metrics)
 logger.log_artifacts([str(checkpoint_path)])
 logger.log_end(success=True)
 except Exception as e:
 logger.log_end(success=False, error=str(e))
 raise
 finally:
 logger.save()
 logger.print_summary()
```

Colab Compatibility

The `RunLogger` is fully Colab-friendly:

Feature	Local	Colab
Git info	<input checked="" type="checkbox"/> Full	<input type="checkbox"/> Skipped (graceful)
System info	<input checked="" type="checkbox"/> Full	<input checked="" type="checkbox"/> <code>is_colab=True</code>
Run directory	<input checked="" type="checkbox"/> Local path	<input checked="" type="checkbox"/> Drive path
Metrics/artifacts	<input checked="" type="checkbox"/> Full	<input checked="" type="checkbox"/> Full

Notebook Usage

```
In orchestration.ipynb
from core import RunLogger, create_run_logger

def create_notebook_logger(track: str, stage: str = None):
 """Create a run logger for notebook-driven runs."""
 logger = create_run_logger(
 track=track,
 stage=stage,
 run_dir=paths.local_extraction / 'runs' /
f'Track_{track[-1].upper()}',
)
 logger.log_config(CONFIG_OVERRIDES)
 logger.log_note(f"Notebook run | Colab={IN_COLAB}")
 return logger

Usage
logger = create_notebook_logger('trackB', 'train')
logger.log_start()
... run training ...
logger.log_metrics({'mAP': 0.42})
logger.log_end(success=True)
logger.save()
```

## Helper Functions

Quick logging for entire runs:

```
from core import log_trackA_run, log_trackB_run, log_trackC_run

Log a complete Track A run
log_path = log_trackA_run(
 stage='stageA',
 config={'k': 6, 'mode': 'yolo'},
 results={'processed': 1000, 'candidates': 6000},
 artifacts=['predictions.json'],
)

Log a complete Track B run
log_path = log_trackB_run(
 config={'epochs': 20},
 metrics={'mAP': 0.45},
 artifacts=['best.pt'],
)
```

## Viewing Run History

```
from core import get_paths

paths = get_paths()

Find latest run for a track
latest = paths.latest_run('Track_B')
print(f"Latest: {latest}")

List all runs
for track_dir in paths.runs_root.iterdir():
 if track_dir.is_dir():
 print(f"\n{track_dir.name}/")
 for run in sorted(track_dir.iterdir()):
 print(f" {run.name}")
```

---

## Smoke Tests

Each track has a smoke test that validates:

- Config loading works
- Required modules import successfully
- Paths resolve correctly

- Model instantiation succeeds (Track B/C)
- Pycache directories exist (indicates prior successful imports)

## Running Smoke Tests

```
From local_extraction directory
cd local_extraction

Activate virtual environment
.\.venv\Scripts\Activate.ps1 # Windows
source .venv/bin/activate # Linux/Mac

Run individual tests
python -m tests.smoke_test_trackA
python -m tests.smoke_test_trackB
python -m tests.smoke_test_trackC

With options
python -m tests.smoke_test_trackC --rates 0.0,0.3,0.5 --cpu-only

JSON output for automation
python -m tests.smoke_test_trackB --json
```

## Expected Output

```
=====
Track B Smoke Test Results
=====
Timestamp: 2025-12-02 14:30:52
Overall: ✓ PASSED
Elapsed: 2.45s

✓ core_imports: Core modules imported successfully
✓ trackb_imports: Track B modules imported successfully
✓ torch_available: PyTorch 2.1.0+cu118, CUDA available
✓ model_instantiation
 ✓ projector: Projector created: input=512, output=256
 ✓ fusion: Fusion created: 2 layers, 8 heads
 ✓ head: Head created: hidden=256, outputs=3
✓ forward_pass: Forward pass successful, logits shape: (2, 8)
✓ pycache_validation: Found 5 __pycache__ directories

=====
```

---

## Usage Guide

## Local Usage

### 1. Setup Environment

```
Navigate to project
cd D:\Thesis\Ego4d-LiteSTA\local_extraction

Activate virtual environment
.\.venv\Scripts\Activate.ps1

Verify Python path
python -c "import sys; print(sys.executable)"
```

### 2. Run Smoke Tests First

```
Validate everything works before running tracks
python -m tests.smoke_test_trackA
python -m tests.smoke_test_trackB
python -m tests.smoke_test_trackC
```

### 3. Edit Configuration

Edit YAML files in `configs/` directory:

```
configs/trackB.yaml - Enable demo mode
demo:
 enabled: true
 max_samples: 20
 epochs: 2
```

### 4. Run Track Scripts

#### Option A: Direct execution (scripts read configs internally)

```
In your track script, add at the top:
import sys
sys.path.insert(0, 'local_extraction')

from core import load_config, apply_config_to_module
```

```
Load config and apply to module toggles
cfg = load_config('trackA')
... use cfg.get('stage_a.k') etc.
```

### Option B: Use config adapter

```
from core import apply_config_to_module
import trackA.trackA_stageA.trackA_stageA as stageA_module

This sets all module-level toggles from YAML
apply_config_to_module(stageA_module, 'trackA', stage='stageA')

Now run as normal
stageA_module.main()
```

### Option C: Override via environment variables

```
$env:EG04D_DEMO_MODE = "true"
$env:EG04D_DEVICE = "cpu"
python trackA/trackA_stageA/trackA_stageA.py
```

## 5. Common Local Workflows

```
Quick demo run
python -c "
from core import load_config
cfg = load_config('trackB', overrides={'demo': {'enabled': True}})
print(f'Demo mode: {cfg.get(\"demo.enabled\")}')
print(f'Epochs: {cfg.get(\"training.epochs\") if not cfg.get(\"demo.enabled\")
else cfg.get(\"demo.epochs\")}')
"

Check paths resolve correctly
python -c "
from core import get_paths
p = get_paths()
print(f'Frames: {p.extracted_frames}')
print(f'Exists: {p.extracted_frames.exists()}')
"
```

## Google Colab Usage

### 1. Open Orchestration Notebook

Upload or open `local_extraction/notebooks/orchestration.ipynb` in Colab.

### 2. Mount Drive and Setup

The notebook automatically handles:

- Mounting Google Drive
- Setting up Python paths
- Installing dependencies
- Detecting GPU availability

```
Cell 1: Environment Setup (auto-runs)
import os, sys
from pathlib import Path

IN_COLAB = 'google.colab' in sys.modules
if IN_COLAB:
 from google.colab import drive
 drive.mount('/content/drive')
 REPO_ROOT = Path('/content/drive/MyDrive/Thesis/Ego4d-LiteSTA')
else:
 REPO_ROOT = Path.cwd()

Add to path
sys.path.insert(0, str(REPO_ROOT / 'local_extraction'))
os.chdir(str(REPO_ROOT / 'local_extraction'))
```

### 3. Configure Your Run

```
Cell: Configuration Overrides
CONFIG_OVERRIDES = {
 'demo': {
 'enabled': True, # Set False for full training
 'max_samples': 10,
 'epochs': 2,
 },
 'runtime': {
 'device': 'auto', # Uses GPU if available
 'num_workers': 2, # Colab has limited workers
 },
}
```

#### 4. Load Track Configs

```
from core import load_config, get_paths

Load with overrides
trackB_cfg = load_config('trackB', overrides=CONFIG_OVERRIDES)

print(f"Training epochs: {trackB_cfg.get('training.epochs')}")
print(f"Demo mode: {trackB_cfg.get('demo.enabled')}")
print(f"Batch size: {trackB_cfg.get('training.batch_size')}")
```

#### 5. Run Smoke Tests

```
Validate before running
from tests.smoke_test_trackA import run_smoke_tests as test_A
from tests.smoke_test_trackB import run_smoke_tests as test_B
from tests.smoke_test_trackC import run_smoke_tests as test_C

for name, test_fn in [('A', test_A), ('B', test_B), ('C', test_C)]:
 success, results = test_fn()
 print(f"Track {name}: {'✓ PASSED' if success else 'X FAILED'}")
```

#### 6. Run Training

```
Enable the track you want to run
RUN_TRACKB_TRAIN = True

if RUN_TRACKB_TRAIN:
 # Import and run with config
 from trackB import trackB_train_loader
 # Training code here...
```

#### 7. Colab-Specific Tips

Aspect	Recommendation
<b>num_workers</b>	Use 2 (Colab has limited CPU)
<b>batch_size</b>	Start with 8, increase if memory allows
<b>checkpoints</b>	Save to Drive ( <code>/content/drive/...</code> )

Aspect	Recommendation
Runtime	Use GPU runtime (T4 or better)
Timeouts	Colab disconnects after ~12h, use checkpoints

## Configuration Reference

### Variable Interpolation

Use `${section.key}` to reference other config values:

```
version: "v2"
paths:
 base: "local_extraction"
 frames: "${paths.base}/${version}/extracted_frames" # Resolves to
local_extraction/v2/extracted_frames
```

### Config Inheritance

Use `_base_` to inherit from another config:

```
trackA.yaml
base: base.yaml # Inherits all settings from base.yaml

stage_a:
 detector: "yolo" # Add/override specific settings
```

### Environment Variables

Override any setting via environment:

```
Format: EGO4D_<SECTION>_<KEY>=value
export EGO4D_DEMO_ENABLED=true
export EGO4D_TRAINING_EPOCHS=100
export EGO4D_RUNTIME_DEVICE=cuda
```

### CLI Overrides

```
python script.py --training.lr=0.0001 --demo.enabled=true
```



# Migration Guide

Migration Status: ☒ COMPLETE

All track scripts have been migrated to use YAML configuration and integrated with the unified run logging system.

## Migrated Files

File	Config	Run Logging	Notes
trackA/trackA_stageA/trackA_stageA.py	<input checked="" type="checkbox"/>	<input checked="" type="checkbox"/>	Loads from <code>configs/trackA.yaml</code>
trackA/trackA_stageB/trackA_stageB.py	<input checked="" type="checkbox"/>	<input checked="" type="checkbox"/>	Loads from <code>configs/trackA.yaml</code>
trackB/trackB_train_loader.py	<input checked="" type="checkbox"/>	<input checked="" type="checkbox"/>	Loads from <code>configs/trackB.yaml</code>
trackB/trackB_eval.py	<input checked="" type="checkbox"/>	<input checked="" type="checkbox"/>	Loads from <code>configs/trackB.yaml</code>
trackC/trackC_pruning.py	<input checked="" type="checkbox"/>	<input checked="" type="checkbox"/>	Loads from <code>configs/trackC.yaml</code>

## How It Works Now

Each script now loads config and creates a run logger at the top:

# Troubleshooting

## Common Issues

Issue	Solution
<code>ModuleNotFoundError: No module named 'core'</code>	Add <code>local_extraction</code> to <code>sys.path</code>
<code>FileNotFoundError: configs/base.yaml</code>	Run from <code>local_extraction</code> directory
<code>YAML parsing error</code>	Check YAML syntax (indentation, quotes)
Config values not applied	Check <code>_base_</code> inheritance chain
Colab paths wrong	Verify Drive mount path matches config

## Debug Commands

```
Print resolved config
from core import load_config
cfg = load_config('trackB')
print(cfg.raw) # Full dict

Check path resolution
from core import get_paths
paths = get_paths()
for attr in ['extracted_frames', 'manifests', 'yolo_labels']:
 p = getattr(paths, attr)
 print(f"{attr}: {p} (exists: {p.exists()})")

Verify environment detection
from core import is_colab, get_device
print(f"Colab: {is_colab()}, Device: {get_device()}")

Test run logger
from core import create_run_logger
logger = create_run_logger(track='test', stage='debug')
logger.log_config({'test': True})
logger.log_start()
logger.log_metrics({'value': 42})
logger.log_end(success=True)
logger.print_summary()

List run history
from core import get_paths
paths = get_paths()
latest = paths.latest_run('Track_B')
print(f"Latest Track B run: {latest}")
```

## Summary

Component	Purpose	Location
YAML Configs	Externalized settings	configs/*.yaml
Config Loader	Parse, interpolate, override	core/config_loader.py
Paths Module	Environment-aware paths	core/paths.py
Config Adapter	Bridge to existing scripts	core/config_adapter.py
Run Logger	Run tracking, metrics, artifacts	core/run_logger.py
Smoke Tests	Validation before runs	tests/smoke_test_*.py
Orchestration	Colab notebook interface	notebooks/orchestration.ipynb
Run Logs	Persisted run history	runs/Track_*/

For questions or issues, check the smoke test output first—it will identify most configuration problems before they affect training.